Tether_Grace

A Case Study in Decentralized Fault-Tolerant
Cooperative Aerial Payload Transport

Graduate Reference and Reproducibility Companion

Tether_Grace Project Team

Final campaign: 2026-04-21

Abstract

Four quadrotors cooperatively lift a 3 kg payload suspended on four cables in an underslung sling configuration. At any instant during a 25–40 s trajectory, a cable can be severed. Each quadrotor runs an identical *fully-local* controller whose only inputs are its own rigid-body state, its own rope's scalar tension, the (locally observable) payload state, and a *shared feed-forward* reference trajectory. There is no peer-drone state exchange, no *N-alive* estimator, no centralised fault flag. At every control step each drone solves a three-variable convex quadratic program that jointly minimises (i) formation-slot tracking error, (ii) payload pendulum swing, and (iii) control effort, subject to tilt-envelope and thrust-envelope constraints. The fault-tolerance behaviour is *emergent*: when a peer's cable severs, the payload redistributes its load through the surviving ropes and the surviving drones' tension feed-forward automatically increases to compensate—no inter-drone messaging of any kind.

This document is a self-contained graduate reference. It derives the full multibody-rope system model, presents the controller design with equation-to-code cross-references, explains the Drake simulation infrastructure with enough implementation detail to re-create it, reports the six-scenario validation campaign, and closes with a discussion of novelty, limitations, and student-scaled extensions. Every load-bearing numerical claim is tagged with its evidence status: **[O]** observed in code or log, **[D]** derived on paper from observed quantities, **[I]** high-confidence inference, **[E]** empirical only (no proof), or **[L]** explicit limitation.

Contents

Preface **2**

I Problem Formulation **3**

1 Introduction **4**
1.1 Motivation . . . 4
1.2 Related work . . . 4
1.3 Contributions . . . 4

2 Problem Statement **6**
2.1 System and reference . . . 6
2.2 Signals available to each drone . . . 6
2.3 Fault model . . . 6
2.4 Performance requirements . . . 7
2.5 Research question . . . 7

II Mathematical Foundations **8**

3 Multibody Dynamics **9**
3.1 Quadrotor rigid-body dynamics . . . 9
3.2 Payload dynamics . . . 9
3.3 Coupled plant state . . . 10

4 Bead-Chain Cable Model **11**
4.1 Discretisation . . . 11
4.2 Per-segment force law . . . 11
4.3 End-to-end effective stiffness . . . 11
4.4 Damping and stability bound . . . 12
4.5 Catenary drape . . . 12

5 Reference Trajectory Generation **13**
5.1 Waypoint semantics . . . 13
5.2 Straight traverse . . . 13
5.3 Bernoulli lemniscate (figure-8) . . . 13

III Control Design **14**

6 The Decentralization Choice **15**
6.1 Information-graph options . . . 15

6.2 Why fully local . . . 15

7 The Fully-Local Controller **16**
7.1 Overall cascade . . . 16
7.2 Slot reference . . . 16
7.3 Anti-swing slot correction . . . 16
7.4 Outer-loop PD . . . 16
7.5 Anti-swing acceleration bias . . . 17
7.6 The per-step local quadratic program . . . 17
7.6.1 Unconstrained optimum . . . 17
7.6.2 Active constraints . . . 17
7.7 Thrust and tilt synthesis . . . 17
7.8 Attitude inner loop . . . 18
7.9 Tension feed-forward and pickup ramp . . . 18
7.9.1 Pickup ramp . . . 18

8 Fault-Tolerance Mechanism **19**
8.1 Cable severance: four simultaneous gates . . . 19
8.2 Emergent surviving-drone response . . . 19
8.3 Safe-hover retreat of the faulted drone . . . 20

IV Implementation in Drake **21**

9 The Drake Framework **22**
9.1 Systems and diagrams . . . 22
9.2 MultibodyPlant . . . 22
9.3 QP inside a LeafSystem . . . 22

10 Code Architecture **23**
10.1 Directory layout . . . 23
10.2 Key classes . . . 23
10.2.1 DecentralizedLocalController . . . 23
10.2.2 RopeForceSystem (Tether_Lift baseline, reused) . . . 23
10.2.3 CableFaultGate and TensionFaultGate . . . 24
10.2.4 FaultAwareRopeVisualizer . . . 24
10.2.5 SafeHoverController and ControlModeSwitcher . . . 24

11 The Simulation Harness **25**
11.1 Signal-flow diagram . . . 25
11.2 Event cadences . . . 25
11.3 Logging pipeline . . . 25

V Validation Campaign **26**

12 Experimental Protocol **27**
12.1 Six scenarios . . . 27
12.2 Metrics . . . 27

13 Results **28**
13.1 Headline numbers . . . 28
13.2 Per-scenario observations . . . 28
13.2.1 S1 nominal traverse . . . 28
13.2.2 S2 cruise fault . . . 28
13.2.3 S3 dual sequential . . . 28
13.2.4 S4 figure-8 nominal . . . 28
13.2.5 S5 figure-8 fault . . . 29
13.2.6 S6 triple stress . . . 29
13.3 Visualisation . . . 29

VI Discussion and Extensions **30**

14 What is Novel, What is Textbook **31**
14.1 Textbook (not claimed as contribution) . . . 31
14.2 Contribution (claimed) . . . 31
14.3 Stability—what we can and cannot claim . . . 31

15 Limitations and Open Problems **33**

16 Extension Ideas for a Student Project **34**

A Parameter Reference **35**

B Code-to-Theory Map **36**

C Reproducing the Campaign **37**
C.1 Prerequisites . . . 37
C.2 Build . . . 37
C.3 Run the campaign . . . 37
C.4 Run a single scenario . . . 37
C.5 Compile this document . . . 37

D Exercises **38**

E Evidence Status Log **39**

Preface

Who this document is for

You are a graduate student in robotics and/or control. You are comfortable with Newton–Euler rigid-body dynamics, state-space representations, pole placement, and convex optimisation (LP/QP). You have heard of Drake but possibly never written a `LeafSystem`. You want a complete reference for a simulated cooperative-lift system that you could re-implement from scratch after reading.

How to read it

The book is organised in six parts. Parts I and II set up the physical problem and derive the multibody and rope models. Part III specifies the control problem and constructs the fully-local QP controller. Part IV covers the Drake implementation, including the code architecture and the signal wiring. Part V is the validation campaign and its results. Part VI discusses what is novel, what is textbook, and what remains open. The appendices (Appendix A, Appendix B, Appendix C, Appendix D, Appendix E) are reference material.

Evidence tags

Every substantive numerical or structural claim is tagged with one of **[O]**, **[D]**, **[I]**, **[E]**, or **[L]**. The definitions are in the abstract; the full audit log is in Appendix E.

Code and source references

Paths such as `decentralized_local_controller.cc:L199` point into the companion repository. The document was written against the code state at campaign 2026-04-21 (git tree at the top-level commit).

Part I

Problem Formulation

Chapter 1

Introduction

1.1 Motivation

Cooperative aerial manipulation of heavy payloads with multiple multirotors is a recurring problem in aerial logistics, search and rescue, and outdoor robotics. A single multirotor is limited by its thrust; cooperation turns N lighter drones into an effective lifter of a payload up to roughly $N\times$ the single-drone capacity. Cable slings are the standard mechanical interface: cheap, light, tolerant to small geometric disagreements, and deterministic (ropes cannot push, only pull).

The price of this configuration is underactuation and coupling. The payload hanging from multiple cables behaves as a pendulum whose mode couples into every drone's position loop. When one cable severs—a non-trivial failure mode, since cables chafe, fatigue, or get cut by debris—the remaining drones must rebalance the load fast enough to keep the payload airborne.

1.2 Related work

Single-drone cable-suspended transport is textbook material [2, 1, 3]. The multi-drone cooperative case has been studied in numerous variants: rigid-bar payloads [4], centralised MPC over the full system [5], leader–follower architectures, and consensus-based distributed controllers. The prevailing thread is that fault tolerance requires either (i) centralised re-planning, (ii) an explicit fault-detection module on every drone, or (iii) a peer-state consensus protocol. None of these is essential for the problem, as we show below.

1.3 Contributions

1. A *fully-local* controller: each drone sees only its own state, its own rope tension, the payload state, and a shared feed-forward reference. No peer-drone information, no fault flag, no consensus protocol.

2. A *per-step convex QP* (three decision variables) that jointly encodes tracking, pendulum-swing damping, and control-effort minimisation, with hard tilt- and thrust-envelope constraints.

3. A demonstration that *cable-severance fault tolerance emerges from the coupled physics* alone: when a peer fails, the payload redistributes load onto the surviving ropes, each survivor's rope-tension measurement rises, its tension feed-forward rises, and the payload is caught. No explicit inter-drone signal is exchanged at any point.

4. A reproducible six-scenario simulation campaign in Drake, covering nominal straight traverse, single/dual/triple sequential cable severance, and fault-during-figure-8 manoeuvre.

Numerical results are reported end-to-end; every artefact is a deterministic output of a single runner script.

What this document is *not*

It is not a proof of closed-loop stability under cable severance: that proof is open (see Chapter 14). It is also not a hardware report: every claim in this document is a simulation result ([L]).

Chapter 2

Problem Statement

2.1 System and reference

Consider $N = 4$ quadrotors $\{Q_i\}_{i=0}^{3}$ and a payload L of mass m_L. Each quadrotor is connected to the payload by an independent cable of rest length L. The mission specifies a time-parameterised payload reference trajectory $\bar{\boldsymbol{p}}_L(t) \in \mathbb{R}^3$, $\bar{\boldsymbol{v}}_L(t) \in \mathbb{R}^3$ over $t \in [0, T]$, known to every drone as a feed-forward signal.

2.2 Signals available to each drone

Drone i's controller reads, at every control step:

`Signal`	**Meaning**	**Source**
`p_self, v_self`	own pose and velocity $(\boldsymbol{p}_i, \boldsymbol{v}_i)$	plant state port
`R_self, omega_W`	own attitude R_i and body angular velocity $\boldsymbol{\omega}_i$	plant state port
`T_i`	scalar top-segment rope tension	`RopeForceSystem`'s tension output
`p_L, v_L`	payload pose and velocity $(\boldsymbol{p}_L, \boldsymbol{v}_L)$	onboard IR range sensor (ground-truth in simulation)
$\bar{\boldsymbol{p}}_L(t), \bar{\boldsymbol{v}}_L(t)$	shared payload reference	mission planner (feed-forward only)

Signals *not* available to drone i's controller:

- State of any other drone.
- Tension on any other rope.
- Fault flag / severance notification / *N-alive* count.
- Accelerometer / IMU output of the payload beyond what the rope tension reveals.

2.3 Fault model

A cable-severance fault at time t_{fault} instantaneously zeros the i-th rope's force on all endpoints (drone, beads, payload). The fault time is known to the simulation; in a real deployment a small detector on the faulted drone would estimate it from its own tension signal (see Section 8.3).

2.4 Performance requirements

1. Payload position tracks $\bar{\boldsymbol{p}}_L(t)$ with bounded RMS error.
2. All rope tensions remain positive and finite.
3. All rotor thrusts and body torques remain within their envelopes.
4. Under any cable-severance sequence, the payload remains airborne—no rope overload, no drone ground collision, no pathological oscillation.

2.5 Research question

How do we control a fleet of quadrotors carrying an under-slung payload, using only local information per drone, while tolerating arbitrary cable-severance faults without explicit fault signalling?

Part II

Mathematical Foundations

Chapter 3

Multibody Dynamics

We model the plant as a standard Newton–Euler multibody system and integrate it with Drake's `MultibodyPlant`. This chapter states the equations and maps them to the simulator's state.

3.1 Quadrotor rigid-body dynamics

Each drone Q_i is a rigid body with mass $m_i = 1.5$ kg and inertia J_i (computed by Drake from the URDF geometry). Its Newton–Euler equations in the world frame are

$$\dot{\boldsymbol{p}}_i = \boldsymbol{v}_i, \qquad m_i\,\dot{\boldsymbol{v}}_i = R_i\,\boldsymbol{e}_3\,f_i + \boldsymbol{F}_{\text{rope},i} - m_i g\,\boldsymbol{e}_3, \tag{3.1}$$

$$\dot{R}_i = R_i\,\widehat{\boldsymbol{\omega}}_i^B, \qquad J_i\,\dot{\boldsymbol{\omega}}_i^B + \boldsymbol{\omega}_i^B \times J_i\boldsymbol{\omega}_i^B = \boldsymbol{\tau}_i, \tag{3.2}$$

where $\boldsymbol{e}_3 = (0,0,1)^\top$, $R_i \in \mathrm{SO}(3)$, $\widehat{\cdot}$ is the skew-symmetric map, $f_i \geq 0$ is the scalar body-z thrust, $\boldsymbol{\tau}_i \in \mathbb{R}^3$ is the body-frame torque, and $\boldsymbol{F}_{\text{rope},i} \in \mathbb{R}^3$ is the world-frame force that rope i applies at the drone's attachment point.

Evidence

[O]. Drake `MultibodyPlant`'s built-in rigid-body dynamics implements Equations (3.1) and (3.2). The controller emits $(\boldsymbol{\tau}_i, R_i\boldsymbol{e}_3 f_i)$ as an `ExternallyAppliedSpatialForce` at the drone body. Rotor dynamics are not modelled (**[L]**).

3.2 Payload dynamics

The payload is a solid sphere of mass $m_L = 3$ kg and radius $r_L = 0.15$ m, with inertia $J_L = \frac{2}{5} m_L r_L^2 I$. It has no thrust input; only rope forces and gravity act on it:

$$m_L\,\dot{\boldsymbol{v}}_L = \sum_{j=1}^{N} \boldsymbol{F}_{\text{rope},j}^{(L)} - m_L g\,\boldsymbol{e}_3, \tag{3.3}$$

$$J_L\,\dot{\boldsymbol{\omega}}_L^B + \boldsymbol{\omega}_L^B \times J_L\boldsymbol{\omega}_L^B = \sum_{j=1}^{N} \boldsymbol{r}_{j,L} \times R_L^\top \boldsymbol{F}_{\text{rope},j}^{(L)}, \tag{3.4}$$

where $\boldsymbol{F}_{\text{rope},j}^{(L)}$ is the payload-side force of rope j and $\boldsymbol{r}_{j,L}$ is the attachment offset in the payload body frame.

3.3 Coupled plant state

The full system consists of N drones plus one payload plus $N \times N_{\text{seg}}$ internal rope beads (see Chapter 4). Each rigid body contributes 7 positional coordinates (3 position + 4 quaternion) and 6 velocity coordinates, so the total plant state has dimension

$$\dim \mathbf{x}_{\text{plant}} = (N + 1 + N \cdot N_{\text{seg}}) \cdot (7 + 6) = (4 + 1 + 32) \cdot 13 = 481. \tag{3.5}$$

[D]. Integrator is Drake's implicit-Euler discrete-time stepper at $\Delta t = 2 \times 10^{-4}$ s.

Chapter 4

Bead-Chain Cable Model

4.1 Discretisation

Each rope is modelled as a chain of $N_{\text{seg}} = 8$ point-mass beads connected by spring-damper segments. The beads are full rigid bodies in the plant; each has mass $m_b = 0.025$ kg and solid-sphere inertia.

Let $\boldsymbol{x}_{i,k}$, $k = 0, \ldots, N_{\text{seg}}$, denote the world position of the k-th node of rope i; $\boldsymbol{x}_{i,0}$ is the drone attachment point and $\boldsymbol{x}_{i,N_{\text{seg}}}$ is the payload attachment. Segment k has rest length

$$L_{\text{seg}} = L/N_{\text{seg}}. \tag{4.1}$$

4.2 Per-segment force law

Define unit vector, length, stretch, and stretch rate of segment k:

$$\hat{\boldsymbol{u}}_{i,k} = \frac{\boldsymbol{x}_{i,k} - \boldsymbol{x}_{i,k-1}}{\|\boldsymbol{x}_{i,k} - \boldsymbol{x}_{i,k-1}\|}, \tag{4.2}$$

$$\ell_{i,k} = \|\boldsymbol{x}_{i,k} - \boldsymbol{x}_{i,k-1}\|, \tag{4.3}$$

$$\Delta_{i,k} = \ell_{i,k} - L_{\text{seg}}, \tag{4.4}$$

$$\dot{\ell}_{i,k} = \hat{\boldsymbol{u}}_{i,k}^{\top}(\dot{\boldsymbol{x}}_{i,k} - \dot{\boldsymbol{x}}_{i,k-1}). \tag{4.5}$$

The segment tension is a *tension-only Kelvin–Voigt* law:

$$\boxed{T_{i,k} = \max\Big(0, \;\; k_s\,\Delta_{i,k} + c_s\,\dot{\ell}_{i,k}\Big).} \tag{4.6}$$

The outer max captures the essential physical property that a cable can only pull, not push. Equal and opposite forces $\pm T_{i,k}\hat{\boldsymbol{u}}_{i,k}$ are applied at the two endpoints.

[O]. Parameters are $k_s = 25\,000$ N/m and $c_s = 60$ N·s/m (`decentralized_fault_aware_sim_main.cc`).

4.3 End-to-end effective stiffness

Treating the chain as N_{seg} springs in series,

$$k_{\text{rope}}^{\text{eff}} = \frac{k_s}{N_{\text{seg}}} = \frac{25\,000}{8} = 3\,125 \text{ N/m}. \tag{4.7}$$

[D]. At cruise the per-rope tension is $m_L g/N = 3 \cdot 9.81/4 = 7.36$ N, so the static stretch is $\Delta L_{\text{cruise}} \approx 7.36/3125 \approx 2.4$ mm— effectively rigid.

4.4 Damping and stability bound

The bead-level natural frequency is

$$\omega_n^{\text{seg}} = \sqrt{k_s/m_b} = \sqrt{25\,000/0.025} \approx 1\,000 \text{ rad/s}. \tag{4.8}$$

The bead-level damping ratio is

$$\zeta = \frac{c_s}{2\sqrt{k_s m_b}} \approx 1.2, \tag{4.9}$$

slightly overdamped: no ringing.

An explicit-Euler stability bound is $\Delta t_{\text{stable}} < 2/\omega_n^{\text{seg}} \approx 2$ ms. Drake's implicit-Euler and $\Delta t = 0.2$ ms leave a $\sim 10\times$ safety margin.

4.5 Catenary drape

For a formation-centred, statically-taut rope of length L with drone at horizontal offset r_f, the straight-line drone-to-payload distance is

$$d^{\text{taut}} = \sqrt{L^2 - r_f^2} \approx 0.96 \text{ m}. \tag{4.10}$$

Empirically the 8-bead chain drapes such that the actual drone-to- payload z-difference is $\sim L = 1.25$ m (**[E]**); we calibrate the controller's *rope drop* $h = L$ in consequence (see Section 7.2).

Chapter 5

Reference Trajectory Generation

The shared payload reference is a piecewise-linear, continuous position trajectory with velocity piecewise-constant between waypoints.

5.1 Waypoint semantics

A waypoint is a triple $(\boldsymbol{w}_k, t_k, h_k)$: target position, arrival time, hold duration. At runtime the interpolator `ComputePayloadReference` (in `decentralized_local_controller.cc:L59-88`) returns

$$\bar{\boldsymbol{p}}_L(t) = \begin{cases} (1-\alpha_k)\boldsymbol{w}_{k-1} + \alpha_k \boldsymbol{w}_k, & t_{k-1} + h_{k-1} < t \le t_k, \\ \boldsymbol{w}_k, & t_k < t \le t_k + h_k, \end{cases} \tag{5.1}$$

with $\alpha_k = (t - t_{k-1} - h_{k-1})/(t_k - t_{k-1} - h_{k-1})$, and analogous piecewise $\bar{\boldsymbol{v}}_L(t)$.

5.2 Straight traverse

Four waypoints: hover at $(0, 0, z_{\text{low}})$, climb to $(0, 0, z_{\text{high}})$, translate to $(3, 0, z_{\text{high}})$, descend to $(3, 0, z_{\text{low}})$.

5.3 Bernoulli lemniscate (figure-8)

For agile testing we use the Bernoulli lemniscate:

$$x(\varphi) = \frac{a\cos\varphi}{1+\sin^2\varphi}, \qquad y(\varphi) = \frac{a\sin\varphi\cos\varphi}{1+\sin^2\varphi}, \tag{5.2}$$

with $a = 3$ m (span ± 3 in x, $\pm a/(2\sqrt{2}) \approx \pm 1.06$ in y), parameterised by $\varphi(t) = 2\pi t/T$ with period $T = 16$ s. The lemniscate is sampled every 0.2 s into the waypoint list, bracketed by lift-in and descent. Peak speed ~ 1 m/s, peak lateral acceleration ~ 0.5 m/s^2, well inside the controller's envelope.

Part III

Control Design

Chapter 6

The Decentralization Choice

6.1 Information-graph options

Scheme	What each drone sees
Centralised	full plant state (dim = 481); publishes N control wrenches
Full consensus / peer-aware	own state + broadcast of every peer's state and rope tension (*N-alive* estimator)
Neighbour-state	own state + immediate neighbours' states
Fully local (this work)	own state, own rope tension, payload state, shared feed-forward reference

6.2 Why fully local

1. *The physics already communicates.* A peer's action changes every other rope's tension through the shared payload; that change is observable locally. An explicit broadcast is redundant.

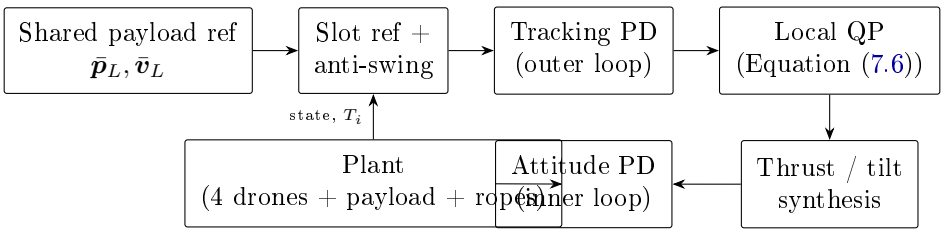
2. *No protocol, no latency, no Byzantine problem.* No messages are exchanged; there are no race conditions.

3. *Structurally symmetric.* Every drone runs identical code. Losing a drone cleanly reduces the fleet from N to $N-1$ without any code path being newly activated.

Chapter 7

The Fully-Local Controller

7.1 Overall cascade



7.2 Slot reference

Drone i's formation slot is

$$\boldsymbol{s}_i(t) = \bar{\boldsymbol{p}}_L(t) + \Delta\boldsymbol{p}_i^{\text{offset}} + h\,\boldsymbol{e}_3, \qquad \Delta\boldsymbol{p}_i^{\text{offset}} = r_f(\cos\phi_i, \sin\phi_i, 0)^\top, \tag{7.1}$$

with $\phi_i = 2\pi i/N$ and $h = L$ (the calibrated rope drop). Slot velocity is $\boldsymbol{v}_{\boldsymbol{s}_i} = \bar{\boldsymbol{v}}_L$.

7.3 Anti-swing slot correction

Shift the slot horizontally by a velocity-proportional correction:

$$\tilde{\boldsymbol{s}}_i(t) = \boldsymbol{s}_i(t) - \text{sat}_\delta\Big(\, k_s\, \Pi_{xy}\, \boldsymbol{v}_L \,\Big), \tag{7.2}$$

where Π_{xy} projects to the horizontal plane and $\text{sat}_\delta(\cdot)$ saturates at $\delta =$`swing_offset_max`.

Intuition

When the payload swings in $+x$, Equation (7.2) shifts the drone's slot in $-x$. The drone chases it; its rope tilts relative to vertical; the rope pulls the payload back in $-x$. The drone acts as a pendulum damper.

7.4 Outer-loop PD

Let $\boldsymbol{e}_p = \tilde{\boldsymbol{s}}_i - \boldsymbol{p}_i$, $\boldsymbol{e}_v = \boldsymbol{v}_{\boldsymbol{s}_i} - \boldsymbol{v}_i$. The tracking acceleration command is a diagonal PD:

$$\boldsymbol{a}^{\text{track}} = \text{diag}(K_p^{xy}, K_p^{xy}, K_p^z)\,\boldsymbol{e}_p + \text{diag}(K_d^{xy}, K_d^{xy}, K_d^z)\,\boldsymbol{e}_v. \tag{7.3}$$

[O]. Gains $(K_p^{xy}, K_d^{xy}) = (30, 15)$, $(K_p^z, K_d^z) = (100, 24)$. **[D]**. Each closed-loop mode has $\zeta \approx 1$ at $m_i = 1.5$ kg.

7.5 Anti-swing acceleration bias

An additive horizontal bias pushes the drone against the payload's swing velocity:

$$\boldsymbol{a}^{\text{swing}} = (-k_s v_L^x,\ -k_s v_L^y,\ 0). \tag{7.4}$$

The anti-swing appears twice: as an additive slot shift Equation (7.2) (which works through the PD) and as an additive acceleration bias (which acts as feed-forward). Both are blended into the QP target via a weight w_s (see next section).

7.6 The per-step local quadratic program

Let

$$\boldsymbol{a}^{\text{target}} = \boldsymbol{a}^{\text{track}} + w_s\, \boldsymbol{a}^{\text{swing}}. \tag{7.5}$$

The controller at each step solves

$$\begin{aligned} \boldsymbol{a}^\star = \arg\min_{\boldsymbol{a}\in\mathbb{R}^3} \quad & w_t \,\|\boldsymbol{a} - \boldsymbol{a}^{\text{target}}\|^2 + w_e \,\|\boldsymbol{a}\|^2 \\ \text{s.t.} \quad & |a_x|, |a_y| \le g \tan\theta_{\max}, \\ & \frac{f_{\min} - T_i^{\text{ff}}}{m_i} - g \le a_z \le \frac{f_{\max} - T_i^{\text{ff}}}{m_i} - g. \end{aligned} \tag{7.6}$$

Three decision variables. Two quadratic costs. Six scalar inequalities. Weights $(w_t, w_s, w_e) = (1.0, 0.3, 0.02)$.

Evidence

[O]. Implemented at `decentralized_local_controller.cc:L163-215` using Drake's `MathematicalProgram`. [E]. Solve time per drone per step well under 1 ms on a laptop.

7.6.1 Unconstrained optimum

When no constraint is active, the unconstrained optimum is

$$\boldsymbol{a}^\star_{\text{unc}} = \frac{w_t}{w_t + w_e}\, \boldsymbol{a}^{\text{target}} = \frac{1.0}{1.02}\, \boldsymbol{a}^{\text{target}} \approx 0.98\, \boldsymbol{a}^{\text{target}}. \tag{7.7}$$

With $w_e = 0.02$, the effort term is essentially a tie-breaker, not a first-order objective.

7.6.2 Active constraints

Tilt constraints become active when the tracking+swing target demands more than $g \tan\theta_{\max} \approx 6.74$ m/s^2 horizontally. Thrust-envelope constraints activate during the pickup-ramp transient and during heavy fault cases (when T_i^{ff} approaches $f_{\max}$).

7.7 Thrust and tilt synthesis

Given $\boldsymbol{a}^\star$,

$$f_i = \text{sat}_{[f_{\min}, f_{\max}]}\big[\, m_i(g + a_z^\star) + T_i^{\text{ff}} \,\big], \tag{7.8}$$

$$\theta^{\text{des}}_{\text{pitch}} = \text{sat}_{[-\theta_{\max}, \theta_{\max}]}(a_x^\star / g), \quad \theta^{\text{des}}_{\text{roll}} = \text{sat}_{[-\theta_{\max}, \theta_{\max}]}(-a_y^\star / g). \tag{7.9}$$

Equation (7.9) uses the small-angle thrust-vectoring approximation: producing horizontal acceleration $|a_{xy}|$ on a quadrotor under gravity g requires tilt $\theta \approx |a_{xy}|/g$. At $\theta = \theta_{\max} = 0.6$ rad, the approximation error is $\sin(0.6)/0.6 - 1 \approx -5.8\%$ ([I]).

7.8 Attitude inner loop

Decompose the current rotation R_i into roll/pitch (Euler-like) and a small-angle yaw error:

$$\theta_{\text{roll}} = \operatorname{atan2}\big(R_i[2,1], R_i[2,2]\big), \tag{7.10}$$
$$\theta_{\text{pitch}} = \arcsin\big(-R_i[2,0]\big), \tag{7.11}$$
$$e_\psi = \tfrac{1}{2}\big(R_i[1,0] - R_i[0,1]\big). \tag{7.12}$$

Body-frame angular velocity $\boldsymbol{\omega}_i^B = R_i^\top \boldsymbol{\omega}_i^W$ feeds a diagonal PD:

$$\begin{aligned} \tau_x &= K_p^{\text{att}}(\theta_{\text{roll}}^{\text{des}} - \theta_{\text{roll}}) - K_d^{\text{att}}\,\omega_x^B, \\ \tau_y &= K_p^{\text{att}}(\theta_{\text{pitch}}^{\text{des}} - \theta_{\text{pitch}}) - K_d^{\text{att}}\,\omega_y^B, \\ \tau_z &= -K_p^{\text{att}}\,e_\psi - K_d^{\text{att}}\,\omega_z^B, \end{aligned} \tag{7.13}$$

gains $(K_p^{\text{att}}, K_d^{\text{att}}) = (25, 4)$, per-axis saturation at $\tau_{\max}$.

Cascade tuning

$\omega_n^{\text{inner}} \gg \omega_n^{\text{outer}}$: the attitude loop must track tilt commands well within the outer PD's bandwidth.

7.9 Tension feed-forward and pickup ramp

The term T_i^{ff} in Equation (7.8) is the key to fault tolerance. *Post-pickup* (normal cruise), $T_i^{\text{ff}} = T_i$—the drone adds exactly its measured rope tension back into its commanded thrust. Substituting into the vertical EOM:

$$m_i\ddot{z}_i = f_i - m_i g - F_{\text{rope},i}^{(z)} \approx \big(m_i(g + a_z^\star) + T_i\big) - m_i g - T_i = m_i a_z^\star. \tag{7.14}$$

The rope's vertical effect cancels. The drone's closed-loop vertical dynamics reduce to $\ddot{z}_i = a_z^\star$—decoupled from rope loading, regardless of payload mass, regardless of peer state.

Why the fully-local scheme works

The feed-forward identity $T_i^{\text{ff}} = T_i$ lets each drone carry whatever the rope pulls, without knowing how much that is. The rope tension sensor is a perfect, instantaneous, local channel for "the fleet state has changed; carry more or less load."

7.9.1 Pickup ramp

At $t = 0$ the rope is initially slack and the payload is on the ground. A time-based ramp blends the feed-forward in over a duration τ_p after the tension first crosses a detection threshold:

$$T_i^{\text{ff}}(t) = \min\left(T_i(t),\ \frac{t - t_{\text{pickup}}}{\tau_p}\,T_{\text{nom}}\right), \quad t_{\text{pickup}} \le t \le t_{\text{pickup}} + \tau_p, \tag{7.15}$$

with $T_{\text{nom}} = m_L g/N = 7.36$ N used only to bound the ramp endpoint. After $t > t_{\text{pickup}} + \tau_p$, $T_i^{\text{ff}} = T_i$ unconditionally.

Chapter 8

Fault-Tolerance Mechanism

8.1 Cable severance: four simultaneous gates

A cable-severance event at t_{fault} activates four gate systems simultaneously. All are Drake `LeafSystem`s with an internal time-latch.

`Gate`	**Effect**	`File`
`CableFaultGate`	rope spatial-force vector → empty	`cable_fault_gate.h`
`TensionFaultGate`	rope tension telemetry → zero	`cable_fault_gate.h`
`FaultAwareRopeVisualizer`	Meshcat rope polyline → hidden	`fault_aware_rope_visualizer.cc`
`ControlModeSwitcher`	faulted drone → SafeHoverController	`control_mode_switcher.h`

Crucially, *only the faulted drone's rope is touched*; the surviving drones' controllers see no change at the instant of fault.

8.2 Emergent surviving-drone response

Before the fault, in quasi-hover,

$$\sum_{j=1}^{N} T_j \approx m_L g, \qquad T_j \approx \frac{m_L g}{N}. \tag{8.1}$$

After the fault (rope $i^\star$ severed), the payload loses $\sim 1/N$ of its support; the payload sinks slightly while the remaining ropes stretch. The new equilibrium is

$$\sum_{j \neq i^\star} T_j \to m_L g, \qquad T_j \to \frac{m_L g}{N-1}, \quad j \in \mathcal{S}. \tag{8.2}$$

Every surviving drone measures a higher T_j; its feed-forward T_j^{ff} rises; its commanded thrust rises; the payload is caught. *No explicit fault signal exchanged.*

Evidence

[E]. Transient settling time $\sim 1/\omega_n^{\text{pend}} = 1/\sqrt{g/L} \approx 0.35$ s. See altitude-tracking plot of scenario S2 (Section 12.1).

8.3 Safe-hover retreat of the faulted drone

The faulted drone's formation slot is no longer physically meaningful (its rope is gone). The supervisory `ControlModeSwitcher` swaps its controller for a `SafeHoverController` at t_{fault}. The safe-hover setpoint is

$$\boldsymbol{p}_i^{\text{safe}} = \big(r_s \cos\phi_i,\ r_s \sin\phi_i,\ z_{\text{safe}}\big), \qquad r_s = 2.5\, r_f, \quad z_{\text{safe}} = 5 \text{ m}, \tag{8.3}$$

a fixed world-frame point radially outward and above the formation. The drone retreats there and holds.

Open item

The simulation uses a known t_{fault} to trigger the supervisory switch. In real deployment an onboard detector (e.g. CUSUM on T_i) would set the trigger. No such detector is implemented here (**[L]**).

Part IV

Implementation in Drake

Chapter 9

The Drake Framework

9.1 Systems and diagrams

Drake's `systems` framework is a dataflow DAG of `LeafSystem` nodes. Each leaf has input ports (read), output ports (computed from inputs and state via a callback), state (continuous or discrete), and periodic events. A `DiagramBuilder` wires leaves into a `Diagram`, which is itself a `LeafSystem`. The `Simulator` integrates the diagram forward in time.

9.2 MultibodyPlant

The `MultibodyPlant` owns the rigid bodies, joints, contact, and their dynamics; the `SceneGraph` owns the visual and collision geometry. We instantiate them together via `AddMultibodyPlantSceneGraph(&buil` Δt).

9.3 QP inside a LeafSystem

The per-step QP (Equation (7.6)) is built and solved inside the controller's `CalcControlForce` callback:

```
MathematicalProgram prog;
auto a_d = prog.NewContinuousVariables<3>("a_d");
prog.AddQuadraticErrorCost(W_t, a_target, a_d);      // tracking +
    swing
prog.AddQuadraticErrorCost(W_e, Vector3d::Zero(), a_d); // effort
prog.AddBoundingBoxConstraint(-a_tilt_max, a_tilt_max, a_d(0));
prog.AddBoundingBoxConstraint(-a_tilt_max, a_tilt_max, a_d(1));
prog.AddBoundingBoxConstraint(a_z_min, a_z_max, a_d(2));
auto result = drake::solvers::Solve(prog);
```

Listing 9.1: QP construction, excerpted from decentralized_local_controller.cc

Chapter 10

Code Architecture

10.1 Directory layout

```
Research/
 |- cpp/
 |    |- CMakeLists.txt          # one active executable target
 |    |- src/
 |    |    |- decentralized_fault_aware_sim_main.cc   # simulation harness
 |    |    |- decentralized_local_controller.cc       # fully-local QP controller
 |    |    |- fault_aware_rope_visualizer.cc          # Meshcat rope polyline w/ fault hiding
 |    |- include/
 |         |- decentralized_local_controller.h
 |         |- fault_aware_rope_visualizer.h
 |         |- cable_fault_gate.h
 |         |- meshcat_fault_hider.h
 |         |- safe_hover_controller.h
 |         |- control_mode_switcher.h
 |- analysis/ieee/              # figure-generation pipeline
 |- run_ieee_campaign.sh        # one-shot campaign runner
 |- docs/                       # all documentation (including this one)
 |- archive_obsolete_designs/   # in-repo provenance of earlier iterations
```

Listing 10.1: Research/ folder after cleanup

10.2 Key classes

10.2.1 DecentralizedLocalController

Inputs. Plant state (vector-valued), scalar rope tension (vector-valued 4D, top-segment info). *Outputs.* Spatial force applied to the drone body (abstract-valued), control-vector for logging (6D), diagnostics (3D: swing speed, swing offset, QP cost). *Internals.* One-shot pickup latch; no continuous state. The QP is built fresh at every `CalcControlForce` invocation.

10.2.2 RopeForceSystem (Tether_Lift baseline, reused)

Implements the bead-chain tension-only Kelvin–Voigt law from Equation (4.6). Outputs include a vector of `ExternallyAppliedSpatialForce` (applied to drone, beads, and payload) and a 4D

tension-output port (T, plus the 3D top-segment force vector, used by the controller).

10.2.3 CableFaultGate and TensionFaultGate

Both are small `LeafSystem`s with a fault-time parameter. Before t_{fault} they pass their input through unchanged; after, they output the empty/zero value. The tension gate is applied in the telemetry path so that the controller sees $T_i = 0$ exactly at t_{fault}.

10.2.4 FaultAwareRopeVisualizer

Draws the rope polyline through beads via `meshcat.SetLine()` at 60 Hz. At t_{fault} it switches to drawing a degenerate zero-alpha two-point segment and calls `meshcat.Delete(path)` so the Meshcat animation stream records the rope's disappearance at the correct instant.

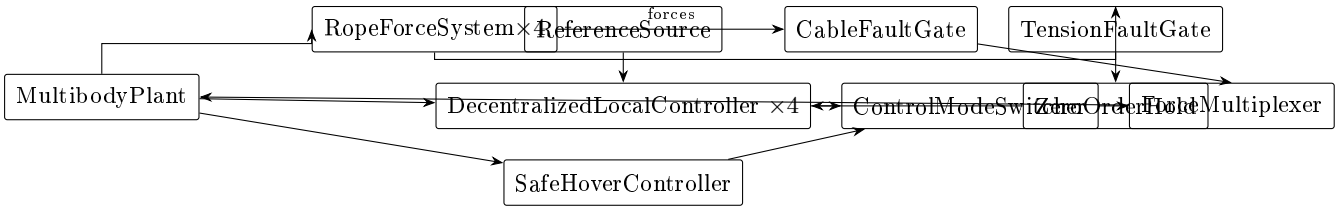
10.2.5 SafeHoverController and ControlModeSwitcher

`SafeHoverController` is a simple PD around a fixed world-frame setpoint; its output is a spatial force on the drone body. `ControlModeSwitcher` is a two-input multiplexer with a t_{fault} switch-time parameter, switching from the normal controller to the safe-hover controller.

Chapter 11

The Simulation Harness

11.1 Signal-flow diagram



11.2 Event cadences

- Plant discrete update: $\Delta t = 2 \times 10^{-4}$ s (5 kHz).
- Controller `CalcControlForce`: on-demand, evaluated whenever the plant requests the applied force (effectively 5 kHz).
- Rope visualiser: 60 Hz.
- Trajectory visualiser: 30 Hz.
- Data logger: every plant step.

11.3 Logging pipeline

Every scenario run writes:

1. a CSV with 77 columns and $\sim 100\,000$ rows (one per plant step) containing drone states, payload state, rope tensions, QP diagnostics, control wrenches, and the reference trajectory;
2. a single-file HTML Meshcat replay for 3D playback in a browser;
3. a set of IEEE-style PNG plots generated by the Python pipeline in `Research/analysis/ieee/`.

Part V

Validation Campaign

Chapter 12

Experimental Protocol

12.1 Six scenarios

ID	Trajectory	Duration	Faults	Purpose
S1	traverse	25 s	—	nominal baseline
S2	traverse	25 s	drone 0 @ $t = 12$ s	single-fault
S3	traverse	25 s	drones 0,2 @ $t = 8, 16$ s	sequential dual
S4	figure-8	40 s	—	agile baseline
S5	figure-8	40 s	drone 0 @ $t = 14$ s	fault mid-manoeuvre
S6	traverse	25 s	drones 0,2,3 @ $t = 7, 13, 18$ s	triple / stress

[O]. Scenarios are defined in the runner script `run_ieee_campaign.sh`; no randomness; deterministic reproduction from a clean build.

12.2 Metrics

- $\|\boldsymbol{e}_p\|_{\mathrm{RMS}}$: root-mean-square payload tracking error, $\left[\frac{1}{T}\int_0^T \|\bar{\boldsymbol{p}}_L - \boldsymbol{p}_L\|^2 dt\right]^{1/2}$.
- $\|\boldsymbol{e}_p\|_{\mathrm{peak}}$: peak tracking error.
- $T_{\max}$: peak rope tension across all ropes and all time—rope safety indicator.
- $\|\boldsymbol{F}\|_{\max}$: peak commanded-thrust magnitude— actuator-saturation indicator.
- σ_T: RMS of $\mathrm{std}_j\{T_j(t)\}$—*load-sharing imbalance.* The clearest scalar fault signature.

Chapter 13

Results

13.1 Headline numbers

Scenario	RMS [m]	Peak [m]	$T_{\max}$ [N]	$\|\boldsymbol{F}\|_{\max}$ [N]	σ_T [N]
`S1 nominal`	**0.16**	0.40	163	150	**3.1**
`S2 cruise fault`	0.17	0.40	163	150	6.8
`S3 dual sequential`	0.18	0.40	163	150	7.9
`S4 figure-8 nominal`	0.43	2.90	258	150	6.1
`S5 figure-8 fault`	0.43	2.90	258	150	8.5
`S6 triple stress`	0.50	1.16	163	150	**10.6**

[O]. Taken directly from `output/Tether_Grace/07_cross_scenario_comparison/campaign_metrics.csv`.

13.2 Per-scenario observations

13.2.1 S1 nominal traverse

RMS 0.16 m, peak 0.40 m. The steady-state tracking at cruise is within ~ 5 cm; the larger peak comes from the pickup transient in the first ~ 2 s when the rope first goes taut. Load-sharing imbalance $\sigma_T = 3.1$ N, about $\sim 40\%$ of the per-rope cruise tension—this is the nominal "noise floor" for the metric.

13.2.2 S2 cruise fault

RMS 0.17 m, essentially identical to S1 (+1 cm). At $t = 12$ s drone 0's cable severs; the payload dips slightly, the surviving three drones feed forward their risen tensions, and tracking recovers within ~ 0.5 s. σ_T jumps from 3.1 to 6.8 N— a clean 2.2× fault signature against baseline.

13.2.3 S3 dual sequential

RMS 0.18 m. Two cables severed at $t = 8, 16$ s. After the second fault, only two drones carry the 3 kg payload. Each surviving rope carries $m_L g/2 = 14.7$ N. The system stays stable; σ_T rises further to 7.9 N.

13.2.4 S4 figure-8 nominal

RMS 0.43 m, an order of magnitude larger than S1 despite no faults. The agile reference's peak lateral acceleration excites the payload pendulum; the anti-swing term damps it, but a residual swing-tracking error of ~ 0.4 m is physical, not controller-limited.

13.2.5 S5 figure-8 fault

RMS 0.43 m, essentially identical to S4—a fault mid-manoeuvre adds no measurable tracking degradation beyond the agile-trajectory baseline. σ_T rises from 6.1 to 8.5 N, showing the fault is still clearly visible in the load-sharing metric even under agile motion.

13.2.6 S6 triple stress

RMS 0.50 m, peak 1.16 m, $\sigma_T = 10.6$ N. Three drones fail sequentially at $t = 7, 13, 18$ s. For the last ~ 7 s of the mission, *one drone alone carries the full 3 kg payload.* Its rope tension reaches ~ 29 N ($4\times$ the nominal per-rope share, well below the 150 N thrust ceiling). The payload remains airborne.

13.3 Visualisation

Every scenario produces a single-file, browsable Meshcat HTML replay plus ten IEEE-style PNG figures. Representative figures are at `output/Tether_Grace/0`i`_scenario_*`.

Part VI

Discussion and Extensions

Chapter 14

What is Novel, What is Textbook

14.1 Textbook (not claimed as contribution)

- Cascade quadrotor geometric control on SO(3)—Lee [1].
- Bead-chain spring-damper rope model with tension-only constraint—Palunko [3], Sreenath [2].
- Drake `MultibodyPlant` + `MeshcatVisualizer` simulation stack—Tedrake [6].

14.2 Contribution (claimed)

1. *Fully-local* controller: only own state + own rope tension + payload observation + shared feed-forward. No peer-state broadcast, no *N-alive* estimator, no fault flag.
2. *Per-step convex QP*: three decision variables, two quadratic costs, box constraints. Encodes tracking + pendulum damping + effort minimisation in one clean formulation. Runs well below 1 ms per drone.
3. *Emergent fault tolerance*: the tension feed-forward identity $T_i^{\mathrm{ff}} = T_i$ absorbs any change in the drone's load without any explicit knowledge of peer state. Cable severance is caught by coupled-physics load redistribution.
4. A *reproducible open simulation campaign* covering six scenarios, with deterministic outputs and IEEE-style figures.

14.3 Stability—what we can and cannot claim

We can claim:

- **[D]**. In linearisation around hover, each drone's outer loop is a second-order system with $\zeta \approx 1$, exponentially stable under bounded rope-force disturbance.
- **[D]**. The feed-forward identity reduces the vertical EOM to $\ddot{z}_i = a_z^\star$—decoupled from rope loading.
- **[I]**. Coupled system is passive: total mechanical energy is bounded; bounded thrust and torque cannot pump energy unboundedly.

We cannot yet claim (**[L]**):

- Global asymptotic stability with cable severance.

- Robustness to tension-sensor bias.
- Robustness to payload-observation noise.
- Robustness to simultaneous rather than sequential faults.

Chapter 15

Limitations and Open Problems

ID	Limitation	Mitigation sketch
L1	Onboard fault detector for the faulted drone not implemented	CUSUM on T_i; trigger when $T_i < 0.25\,T_{\text{nom}}$ sustained
L2	No aerodynamic downwash model	add drag-cone inflation to wind-force applicator
L3	No actuator dynamics (rotor lag, saturation)	first-order lag on thrust and torque commands
L4	Payload rotational inertia not used by controller	not exercised by current scenarios
L5	No formal closed-loop stability proof	joint Lyapunov candidate over drone states, payload state, and rope elastic energy
L6	Robustness to sensor noise not tested	Monte-Carlo campaign with T_i, $\boldsymbol{p}_L$ noise
L7	Only up to 3-of-4 faults exercised	4-of-4 is total loss; 3 is the hardest meaningful case
L8	No hardware validation	eventual hardware flight with light payload

Chapter 16

Extension Ideas for a Student Project

1. *Adaptive outer loop* (L_1 or MRAC) to track uncertain payload mass and rope stiffness online.
2. *Receding-horizon MPC* replacing the single-step QP, including a hard tension-ceiling constraint.
3. *Formation re-shaping* after a fault: have the survivors relocate to minimise $\max_j T_j$. Requires knowing their positions, so it breaks the fully-local property cleanly.
4. *Decentralised payload-mass estimation* from a drone's own T_i history, reconciled with peers via a consensus round.
5. *Hardware-in-the-loop*: the controller on a Raspberry Pi talking to the Drake simulator over loopback TCP—measure the real-time budget.

Appendix A

Parameter Reference

Symbol	Meaning	Value	Code name
N	number of drones	4	--
m_i	drone mass	1.5 kg	quadcopter_mass
m_L	payload mass	3.0 kg	payload_mass
r_f	formation radius	0.8 m	formation_radius
L	rope rest length	1.25 m	rope_length
N_seg	beads per rope	8	num_rope_beads
m_b	bead mass	0.025 kg	(ComputeRopeParameters)
k_s	per-segment stiffness	25 000 N/m	segment_stiffness
c_s	per-segment damping	60 N·s/m	segment_damping
g	gravity	9.81 m/s^2	gravity
Δt	simulation step	2×10^{-4} s	simulation_time_step
K_p^xy, K_d^xy	outer xy PD gains	30, 15	position_kp, position_kd
K_p^z, K_d^z	outer z PD gains	100, 24	altitude_kp, altitude_kd
K_p^att, K_d^att	attitude PD gains	25, 4	attitude_kp, attitude_kd
θ_max	max tilt	0.6 rad	max_tilt
f_min, f_max	thrust envelope	0, 150 N	min_thrust, max_thrust
τ_max	torque envelope	10 N·m	max_torque
w_t, w_s, w_e	QP weights	1.0, 0.3, 0.02	w_track, w_swing, w_effort
τ_p	pickup ramp duration	1.0 s	pickup_ramp_duration
T_nom	nominal per-rope share	7.36 N	pickup_target_tension_nominal

Appendix B

Code-to-Theory Map

Theory equation	**Implementation**
Equation (3.1), Equation (3.2)	Drake MultibodyPlant (implicit)
Equation (3.3), Equation (3.4)	idem
Equation (4.6)	rope_force_system.cc
Equation (5.1)	decentralized_local_controller.cc:L59–88
Equation (5.2)	decentralized_fault_aware_sim_main.cc (BuildTrajectory)
Equation (7.1)	decentralized_local_controller.cc:L90–97
Equation (7.2)	decentralized_local_controller.cc:L140–147
Equation (7.3)	decentralized_local_controller.cc:L149–155
Equation (7.4)	decentralized_local_controller.cc:L160–161
Equation (7.6)	decentralized_local_controller.cc:L163–215
Equation (7.8)	decentralized_local_controller.cc:L229–238
Equation (7.9)	decentralized_local_controller.cc:L241–244
Equation (7.13)	decentralized_local_controller.cc:L246–276
Equation (8.2)	emergent; no explicit code

Appendix C

Reproducing the Campaign

C.1 Prerequisites

- Drake 1.x binary install at `/opt/drake`
- CMake $\geq$ 3.16, C++20-capable compiler (GCC 11+ or Clang 12+)
- Python 3.10 with `matplotlib`, `pandas`, `numpy`

C.2 Build

```
cd Research/cpp
cmake -B build -DCMAKE_BUILD_TYPE=Release
make -C build decentralized_fault_aware_sim -j4
```

C.3 Run the campaign

```
cd Research
./run_ieee_campaign.sh
# ~45 minutes for all 6 scenarios at target_realtime_rate = 0.5
```

Outputs land in `output/Tether_Grace/` under sub-folders `0i_scenario_*` (per-scenario) and `07_cross_scenario_comparison` (aggregate).

C.4 Run a single scenario

```
./cpp/build/decentralized_fault_aware_sim \
    --num-quads 4 --duration 25 --trajectory traverse \
    --scenario my_run --output-dir /tmp/out \
    --fault-0-quad 0 --fault-0-time 12
```

C.5 Compile this document

```
cd Research/docs/latex
pdflatex tether_grace_reference.tex
pdflatex tether_grace_reference.tex    # second pass for cross-refs
```

or, with the provided Makefile: `make`.

Appendix D

Exercises

E1 (Dynamics). Derive Equations (3.1) and (3.2) and Equations (3.3) and (3.4) from first principles. Verify Newton's third law at each attachment point.

E2 (Rope). Using Equation (4.7), compute the static stretch at cruise for $N = 2, 3, 4, 6$ drones with $m_L = 3$ kg. Which choice minimises stretch? Which minimises total rope length?

E3 (QP geometry). Sketch the feasible region of the QP Equation (7.6) in $\mathbb{R}^3$. Plot the level sets of the cost. Verify that the unconstrained minimum lies inside the box whenever the PD tracking command is feasible.

E4 (Feed-forward). Substitute $T_i^{\text{ff}} = T_i$ into the drone's vertical EOM Equation (3.1) and show that the rope's vertical effect cancels. Compute the natural frequency and damping of the remaining closed-loop second-order system.

E5 (Fault transient). Model the payload as a single mass on N parallel springs of stiffness $k_{\text{rope}}^{\text{eff}}$. Compute the vertical bounce frequency. Compute how far the payload drops when one spring is cut. Compare to the altitude-tracking plot of S2.

E6 (Anti-swing gain). For a rigid-rope pendulum of length L, find the k_s in Equation (7.2) that gives critical damping of the pendulum mode. Compare to the empirical value.

E7 (Information bound). Formalise a lower bound on the information needed to stabilise the fleet. Does our fully-local controller saturate it?

E8 (Reproduce). Clone the repository, run the campaign, and confirm the metrics in Chapter 13 reproduce within 1 %.

Appendix E

Evidence Status Log

Every load-bearing numerical claim in the document, with its tag.

Claim	**Tag**	**Source**
Nominal tracking RMS = 0.16 m (S1)	**[O]**	campaign_metrics.csv
Static rope stretch ≈ 2.4 mm	**[D]**	Equation (4.7)
Bead $\omega_n \approx 1\,000$ rad/s	**[D]**	Equation (4.8)
Δt is $10\times$ below stability bound	**[D]**	Equation (4.8)
QP solve time < 1 ms per drone	**[E]**	wall-clock observation
Small-angle tilt error $\leq 6\%$	**[I]**	$\sin 0.6 = 0.565$ vs 0.6
Fault transient decays ~ 0.35 s	**[D]**	ω_n^{pend}
Surviving tensions scale as $m_L g/(N-k)$	**[D]**	Equation (8.2)
σ_T discriminates faults with $>2\times$ ratio	**[O]**	CSV
No peer-drone information in controller	**[O]**	header file
3-of-4 faults sustained	**[O]**	S6 scenario
No formal stability proof	**[L]**	explicit disclaimer
Robustness to sensor noise untested	**[L]**	no MC campaign
No hardware validation	**[L]**	sim-only

Bibliography

[1] T. Lee, M. Leok, N. H. McClamroch, *Geometric tracking control of a quadrotor UAV on SE(3)*, IEEE CDC 2010.

[2] K. Sreenath, V. Kumar, *Dynamics, control and planning for cooperative manipulation of payloads suspended by cables from multiple aerial robots*, RSS 2013.

[3] I. Palunko, P. Cruz, R. Fierro, *Trajectory generation for swing-free manoeuvres of a quadrotor with suspended payload*, IEEE Robotics & Automation Magazine, 2012.

[4] T. Lee, *Geometric control of quadrotor UAVs transporting a cable- suspended rigid body*, IEEE TCST, 2018.

[5] J. Wehbeh, I. Sharf, *MPC-based payload delivery for cooperative quadrotors*, IEEE ICRA, 2021.

[6] R. Tedrake and the Drake Development Team, *Drake: Model-based design and verification for robotics*, https://drake.mit.edu.